Master Technology Learning Checklist

> ***Source:*** *Rebuilt from* `Technology_Checklist.pdf`*,* `DSA_Checklist.pdf`*,* `Interview_Checklist.pdf`*, and a full inspection of all project directories in* `D:\Work`*.* ***Last Updated:*** *October 2026* ***Companion Checklists:*** *DSA →* `D:\Work\Learning Engg\roadmap\checklist\DSA_Checklist.pdf` *| Interview →* `Interview_Checklist.pdf`

How to Use This Checklist

- **Work through phases in order.** Each phase builds on the previous one. DSA (Phase 2) is a parallel track — work it alongside other phases.
- **Priority labels:**
  - `[CORE]` — foundational or practically essential for your primary engineering tracks. Do these first.
  - `[STRETCH]` — advanced depth that can be deferred without blocking foundational competence. Study after CORE items.
  - `[OPTIONAL]` — useful but not necessary for your primary objectives.
- **Project annotations** (e.g., `→ Shard`, `→ Cairn`) indicate where you have already implemented a concept. Use those projects to deepen understanding — prior implementation is not the same as systematic mastery.
- **Verification items** (`□ Verify:`) require you to demonstrate understanding, not just passive reading.
- **Do not mark an item complete** until you can explain the concept clearly, identify where it breaks down, and recognize it in unfamiliar code.
- This checklist is long by design. It is a reference document, not a daily to-do list. Focus on one section at a time.

Phase Priority Overview (Career Tracks)

Phase	Java Backend	Full-Stack	Python Backend	AI Engineering	QA / Testing
Phase 1: Foundations	CORE	CORE	CORE	CORE	CORE
Phase 2: DSA	CORE	CORE	CORE	CORE	CORE
Phase 3: Databases	CORE	CORE	CORE	CORE	CORE
Phase 4: Java / Spring Boot	CORE	CORE	—	STRETCH	—
Phase 5: Python Backend	STRETCH	STRETCH	CORE	CORE	—
Phase 6: Architecture & Design	CORE	CORE	CORE	CORE	—

Phase	Java Backend	Full-Stack	Python Backend	AI Engineering	QA / Testing
Phase 7: Frontend	STRETCH	CORE	—	—	CORE
Phase 8: DevOps & Deployment	CORE	CORE	CORE	CORE	CORE
Phase 9: LLD	STRETCH	STRETCH	STRETCH	STRETCH	OPTIONAL
Phase 10: HLD	CORE	CORE	CORE	CORE	OPTIONAL
Phase 11: Gen AI & AI Engineering	STRETCH	STRETCH	STRETCH	CORE	OPTIONAL
Phase 12: QA & Specialized Testing	OPTIONAL	OPTIONAL	OPTIONAL	OPTIONAL	CORE
Phase 13: Projects & Interview	CORE	CORE	CORE	CORE	CORE

Phase 1: Programming Foundations & Core Languages

> ***Goal:*** *Develop solid, systematic knowledge of the languages and runtime environments you actually use. Prior project experience does not substitute for foundational depth.*

1.1 Environment, Tooling & Workflow `[CORE]`

- [ ] Understand the distinction between JDK, JRE, and JVM; know what each does at runtime
- [ ] Configure and use a terminal, package manager, and shell effectively (PowerShell / Bash)
- [ ] Use Git for version control: staging, branching, merging, rebasing, conflict resolution
- [ ] Understand `.gitignore`, `git log`, `git stash`, `git bisect`, and `git blame`
- [ ] Read and reason about unfamiliar codebases: project structure, dependency graphs, entry points
- [ ] Set up and manage virtual environments (Python `venv` / `poetry`; Java Maven / Gradle)
- [ ] Understand semantic versioning and dependency resolution in `pom.xml`, `pyproject.toml`, `package.json`
- [ ] Use an IDE effectively: breakpoints, step-through debugging, watch expressions, call stacks
- [ ] Understand environment variables, `.env` files, and configuration management across environments
- [ ] □ Verify: Set up a project from scratch, explain the dependency graph, debug a failing startup

1.2 Java — Fundamentals `[CORE]`

> *Reference: `D:\Work\Learning Engg\Java-Practice\`*

- [ ] JVM architecture: class loader, bytecode, interpreter, JIT compilation, execution engine
- [ ] Primitive types vs. reference types; stack vs. heap allocation; boxing/unboxing and the caching pool
- [ ] Variables, literals, operators, and operator precedence; integer overflow and IEEE 754 floating point
- [ ] Control flow: `if-else`, `switch` expressions (Java 14+), loops, `break`/`continue`/`return`
- [ ] Methods: call-by-value semantics, method overloading, `static` vs. instance methods, `varargs`
- [ ] Arrays: one-dimensional, multi-dimensional, `java.util.Arrays` utility methods
- [ ] Strings: immutability, String Constant Pool, `intern()`, `StringBuilder` vs. `StringBuffer`, common methods
- [ ] Recursion: base case, call stack mechanics, tail recursion limitations in Java

1.3 Java — Object-Oriented Programming [CORE]

> *Reference:* `D:\Work\Learning Engg\Java-Practice\src\com\OOP\`

- [ ] Classes and objects: instance fields, constructors (no-arg, parameterized, chaining with `this()`/`super()`)
- [ ] Access modifiers: `private`, package-private, `protected`, `public` — understand visibility matrix precisely
- [ ] Encapsulation: data hiding, getter/setter patterns, invariant validation
- [ ] Inheritance: `extends`, `super`, method overriding, `@Override`, constructor chaining
- [ ] Polymorphism: static dispatch (overloading) vs. dynamic dispatch (method overriding at runtime)
- [ ] Abstraction: `abstract` classes vs. `interface`; when to use each
- [ ] Interfaces: `default` methods, `static` interface methods, functional interfaces (`@FunctionalInterface`)
- [ ] `final` keyword on classes, methods, and variables
- [ ] `this` reference, `super` reference, anonymous classes, inner classes, static nested classes
- [ ] Composition vs. inheritance: know when inheritance is inappropriate; favor composition
- [ ] Immutability pattern: design an immutable class correctly (`final` fields, defensive copies, no setters)
- [ ] Verify: Design a small OOP model (e.g., ATM, parking lot) and justify each design choice

1.4 Java — Advanced Java [CORE]

> *Reference:* `D:\Work\Learning Engg\Java-Practice\src\com\AdvancedJava\`

Exception Handling

- [ ] `Throwable` hierarchy: `Error` vs. `Exception`; checked vs. unchecked exceptions
- [ ] `try-catch-finally`, `try-with-resources` (AutoCloseable), multi-catch
- [ ] Creating custom exception classes; when to extend `RuntimeException` vs. `Exception`
- [ ] Exception chaining (`initCause`, constructor with cause); common anti-patterns (swallowing exceptions)

Java Collections Framework (JCF) [CORE]

- [ ] Collection hierarchy: `Collection` → `List`, `Set`, `Queue`; `Map` (separate hierarchy)
- [ ] `ArrayList` vs. `LinkedList`: internal structure, time complexity of operations, when to use each
- [ ] `ArrayDeque`: preferred over `Stack` class; use as both stack and queue
- [ ] `HashMap` internals: hash function, bucket array, linked list / tree bin (Java 8+), load factor, resizing
- [ ] `HashMap` vs. `LinkedHashMap` vs. `TreeMap`: ordering guarantees and use cases
- [ ] `HashSet` vs. `LinkedHashSet` vs. `TreeSet`
- [ ] `equals()` and `hashCode()` contract: why violating it breaks HashMap/HashSet behavior
- [ ] `Comparable` vs. `Comparator`: natural ordering vs. custom ordering; sorting with lambdas
- [ ] `PriorityQueue`: min-heap by default, custom comparators for max-heap
- [ ] `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue` — thread-safe collections
- [ ] Iteration: `Iterator`, `ListIterator`, enhanced for-loop, `removeIf`, fail-fast vs. fail-safe iterators
- [ ] □ Verify: Explain why `HashMap` requires both `equals()` and `hashCode()` when using custom keys

Generics

- [ ] Generic classes, generic methods, type parameters and bounds (`<T extends Comparable<T>>`)
- [ ] Wildcards: `?`, `? extends T` (upper bound), `? super T` (lower bound); PECS principle
- [ ] Type erasure: what it means for runtime behavior; why you cannot do `new T[10]`

Functional Programming

- [ ] Lambda expressions: syntax, target typing, effectively final variables in capture
- [ ] Functional interfaces: `Predicate<T>`, `Function<T,R>`, `Consumer<T>`, `Supplier<T>`, `BiFunction`
- [ ] Method references: static, instance, constructor (`ClassName::new`)
- [ ] Streams API: `filter`, `map`, `flatMap`, `reduce`, `collect`, `findFirst`, `anyMatch`, `sorted`, `distinct`, `limit`
- [ ] Terminal vs. intermediate operations; lazy evaluation; short-circuit behavior
- [ ] `Optional<T>`: purpose, `orElse`, `orElseGet`, `orElseThrow`, `map`, `filter`, `ifPresent`; avoid misuse
- [ ] □ Verify: Rewrite a nested loop using Streams; explain when Streams are *not* the right tool

Multithreading & Concurrency [CORE]

- [ ] Thread creation: `Thread` class, `Runnable`, `Callable`, thread lifecycle states
- [ ] `synchronized` blocks and methods: intrinsic locks, monitor pattern
- [ ] `volatile` keyword: memory visibility guarantee; why it does not ensure atomicity for compound operations
- [ ] `java.util.concurrent.locks`: `ReentrantLock`, `ReadWriteLock`, `StampedLock`; advantages over `synchronized`
  - → *Cairn uses `ReentrantReadWriteLock` and `ReentrantLock` for fine-grained cache segment locking*
- [ ] `Executor` framework: `ThreadPoolExecutor`, `Executors` factory, thread pool sizing heuristics

- [ ] `Future` and `CompletableFuture`: chaining (`thenApply`, `thenCompose`, `thenCombine`), exception handling
- [ ] Java 21 Virtual Threads (Project Loom): how they differ from platform threads, when they help
- [ ] Atomic operations: `AtomicInteger`, `AtomicLong`, `AtomicReference`, `LongAdder`
  - → *Cairn uses `LongAdder` for lock-free hit/miss counters*
- [ ] `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `Phaser`
- [ ] Thread-safe collections: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `LinkedBlockingQueue`
- [ ] Race conditions, deadlocks, livelocks, starvation: detection and prevention strategies
- [ ] Double-checked locking pattern: when it is correct (with `volatile`); when it fails
  - → *Cairn uses double-checked locking on passive TTL expiry to avoid write-lock on every read*
- [ ] The Java Memory Model (JMM): happens-before relationship, memory visibility, cache coherence basics

JVM Internals & Memory `[CORE]`

- [ ] JVM memory areas: Heap (Young/Old Gen), Metaspace (replaces PermGen), Stack, PC Register, Native Stack
- [ ] Garbage collection: generational GC hypothesis, Minor GC vs. Full GC, GC roots, mark-sweep-compact
- [ ] G1, ZGC, Shenandoah: purpose and when each is appropriate
- [ ] String interning and the String Constant Pool
- [ ] `OutOfMemoryError` vs. `StackOverflowError`: causes and common scenarios
- [ ] JVM tuning flags: `-Xms`, `-Xmx`, `-XX:+UseG1GC` (conceptual awareness)
- [ ] □ Verify: Trace what happens to memory when a large HashMap is created, filled, and the reference dropped

1.5 Python — Foundations through Intermediate `[CORE]`

> *Reference: `D:\Work\Learning Engg\Python-ChapterWise\` (Chapters 1–8: Variables, Strings, Lists/Tuples, Dictionaries/Sets, Conditionals, Loops, Functions & Recursion); used in Shard, Vigil, Phoenix, Warden, Foundry, StudyLink*

- [ ] Python execution model: interpreter, bytecode, `*.pyc`, CPython vs. other implementations
- [ ] Built-in types: `int`, `float`, `bool`, `str`, `bytes`, `None`; mutable vs. immutable objects
- [ ] Collections: `list`, `tuple`, `dict`, `set`, `frozenset`; when to use each
- [ ] Comprehensions: list, dict, set, and generator expressions
- [ ] Functions: `*args`, `**kwargs`, default arguments, keyword-only arguments, closures, `nonlocal`
- [ ] Decorators: how they work under the hood; `functools.wraps`; common use cases
- [ ] Context managers: `with` statement, `__enter__` / `__exit__`, `contextlib.contextmanager`
- [ ] Iterators and generators: `__iter__` / `__next__`, `yield`, `yield from`
- [ ] Exception handling: `try-except-else-finally`, exception chaining (`raise X from Y`)
- [ ] Modules and packages: `import`, `from ... import`, `__init__.py`, `__all__`, relative imports
- [ ] The CPython GIL: what it is, what it protects, what it does NOT protect

  - → *Shard explores GIL behavior in depth: compound operations are not atomic even with the GIL*
- [ ] Bytecode-level thread safety: why `dict` operations are not always atomic across compound statements
- [ ] `threading.Lock`, `threading.RLock`, `threading.Thread`, `queue.Queue`, `threading.Event`
  - → *Shard uses a coarse-grained `threading.Lock` for cache engine; understand why fine-grained striping adds overhead under GIL*
- [ ] `asyncio`: event loop, `async def`, `await`, coroutines vs. threads vs. processes
- [ ] `multiprocessing` module: bypassing the GIL for CPU-bound work
- [ ] Type hints and annotations: `Optional`, `Union`, `List`, `Dict`, `Tuple`, `TypeVar`, `Protocol`
- [ ] Python packaging: `pyproject.toml`, `poetry`, `pip`, virtual environments, `requirements.txt`

1.6 JavaScript & TypeScript [CORE]

> *Reference: `D:\Work\Learning Engg\JS-for-React\`; used in Aegis, Check-Mate Analyse, Trajectory (frontend), Foundry (frontend), Conclave (frontend), Pulse (k6 scripts), VidyaLink*

- [ ] JavaScript execution model: single-threaded event loop, call stack, task queue, microtask queue
- [ ] `var` vs. `let` vs. `const`: scoping, hoisting, temporal dead zone
- [ ] Closures: what they capture; common pitfall with loops and `var`
- [ ] Prototypes and the prototype chain; `Object.create`, `Object.assign`
- [ ] ES6+ features: arrow functions, template literals, destructuring, rest/spread, optional chaining, nullish coalescing
- [ ] Promises: `.then()`, `.catch()`, `.finally()`, `Promise.all`, `Promise.race`, `Promise.allSettled`
- [ ] `async`/`await`: syntactic sugar over Promises; error handling with `try-catch`
- [ ] Modules: `import`/`export`, named vs. default exports, dynamic `import()`
- [ ] DOM APIs: `querySelector`, `addEventListener`, event bubbling and capturing
- [ ] `fetch` API and `XMLHttpRequest`; `AbortController` for cancellation
- [ ] Web Workers: dedicated workers, `postMessage` / `onmessage`, `SharedArrayBuffer` caveats
  - → *Check-Mate Analyse runs Stockfish 16 WASM in a dedicated Web Worker; understand UCI command queueing and interrupt dominance*
- [ ] WebAssembly (WASM): what it is, how JS loads and calls WASM modules, shared memory model
  - → *Check-Mate Analyse loads `stockfish.wasm` and requires Cross-Origin-Opener-Policy headers for WASM thread support*
- [ ] `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy`: why required for `SharedArrayBuffer` and high-performance WASM

TypeScript [CORE]

- [ ] Type system: primitive types, `any`, `unknown`, `never`, `void`, `undefined` vs `null`
- [ ] Interfaces vs. type aliases; `extends` vs. `&` intersection types; union types `|`
- [ ] Generics: `<T>`, constraints (`extends`), default type parameters

- [ ] Utility types: `Partial<T>`, `Required<T>`, `Readonly<T>`, `Pick<K>`, `Omit<K>`, `Record<K,V>`, `ReturnType`, `Parameters`
- [ ] Type narrowing: `typeof`, `instanceof`, discriminated unions, type guards
- [ ] Declaration files (`.d.ts`) and `@types` packages
- [ ] `tsconfig.json` key options: `strict`, `target`, `module`, `paths`, `esModuleInterop`
- [ ] Verify: Write a generic utility function in TypeScript and explain the type inference

1.7 SQL [CORE]

> *Reference: Used in PostgreSQL across Cairn, Shard, Phoenix, Vigil, Trajectory, StudyLink, Conclave, Foundry*

- [ ] Relational model: tables, rows, columns, primary keys, foreign keys, constraints (`NOT NULL`, `UNIQUE`, `CHECK`)
- [ ] `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, `HAVING`, `LIMIT`/`OFFSET`
- [ ] JOINs: `INNER`, `LEFT`, `RIGHT`, `FULL OUTER`, `CROSS`; understand their output differences
- [ ] Aggregations: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`; window functions (`ROW_NUMBER`, `RANK`, `LEAD`/`LAG`)
- [ ] Subqueries: correlated vs. non-correlated; `EXISTS`, `IN`, `NOT IN`
- [ ] Common Table Expressions (CTEs): `WITH`, recursive CTEs
- [ ] Transactions: `BEGIN`, `COMMIT`, `ROLLBACK`; ACID properties
- [ ] Isolation levels: `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`; when each matters
- [ ] Indexes: B-tree, Hash, composite indexes, partial indexes, covering indexes; `EXPLAIN`/`EXPLAIN ANALYZE`
- [ ] `SELECT FOR UPDATE` (pessimistic locking) and optimistic locking
  - → *Shard (Python), StudyLink, and Conclave use `SELECT FOR UPDATE` for state machine concurrency*
- [ ] Schema design: normalization (1NF, 2NF, 3NF, BCNF), denormalization trade-offs
- [ ] PostgreSQL-specific: `JSONB`, `ARRAY`, `ENUM` types, `pgvector` extension, `pg_isready`, `pg_dump`
  - → *Phoenix, StudyLink use pgvector with HNSW cosine index for vector similarity search*
- [ ] Flyway/Alembic migrations: versioned migration files, `upgrade head`, `validate`
  - → *Trajectory uses Flyway with PostgreSQL native ENUM types; Vigil uses Alembic*
- [ ] Verify: Write a query using CTEs and window functions; explain its `EXPLAIN ANALYZE` output

1.8 Bash / Shell Scripting [CORE]

> *Core-CS Reference: `D:\Work\Learning Engg\Core-CS-interview\CS\os-mastery-cheatsheet.pdf` (Processes, threads, scheduling, virtual memory, paging, inter-process communication)*

- [ ] File system navigation, redirection (`>`, `>>`, `|`), and piping
- [ ] Variables, conditionals (`if`, `case`), loops (`for`, `while`), functions

- [ ] Common utilities: `grep`, `sed`, `awk`, `find`, `xargs`, `curl`, `jq`
- [ ] Environment variables and `.env` file sourcing
- [ ] Process management: `ps`, `kill`, `nohup`, background jobs (`&`, `jobs`)
- [ ] Script permissions (`chmod +x`) and shebang lines
- [ ] Basic automation use cases: build scripts, health check scripts, Docker startup scripts

---

Phase 2: Data Structures & Algorithms (Parallel Track)

> ***This is a parallel track.*** *Work through it alongside other phases, 1-2 problems daily. The full curriculum is in* `D:\Work\Learning Engg\Data-Structures-and-Algorithms\README.md` *and* `DSA_Checklist.pdf`*. This phase summarizes the key coverage and adds depth guidance.*

2.1 Data Structure Foundations `[CORE]`

> *Reference:* `D:\Work\Learning Engg\Data-Structures-and-Algorithms\` *(Java)*

- [ ] **Arrays & Strings:** Index-based access O(1), insertion/deletion O(n); StringBuilder vs. String concatenation
- [ ] **Stacks & Queues:** LIFO vs. FIFO; use `ArrayDeque` in Java (not legacy `Stack`); implement with arrays
- [ ] **Linked Lists (Singly & Doubly):** Node structure, pointer manipulation, sentinel/dummy nodes
  - → *Cairn LRU and Shard LRU implement doubly-linked lists with sentinel head/tail nodes*
- [ ] **Hash Maps & Sets:** Hash function, collision resolution (chaining, open addressing), load factor, O(1) amortized
- [ ] **Trees:** BST properties, insertion/search/deletion; AVL trees (concept); Red-Black trees (concept)
- [ ] **Heaps:** Min-heap / max-heap; heap property; `PriorityQueue` in Java; `heapify`, `heappush`, `heappop`
- [ ] **Graphs:** Adjacency list vs. adjacency matrix; directed vs. undirected; weighted vs. unweighted
- [ ] **Trie:** Prefix tree; character-indexed children; search and insert in O(L) where L = word length
- [ ] **Java Collections Framework for DSA:** `ArrayList`, `ArrayDeque`, `HashMap`/`HashSet`, `PriorityQueue`, `TreeMap`
  - → *Shard LFU uses* `dict[int, OrderedDict[str, bool]]` *for O(1) frequency-bucket eviction*

2.2 Algorithmic Patterns `[CORE]`

> *Reference: `D:\Work\Learning Engg\Data-Structures-and-Algorithms\README.md` (Phase 2 Pattern Matrix)*

- [ ] **Two Pointers:** Sorted arrays/strings; converging pointers; `Two Sum II`, `3Sum`, `Sort Colors`
- [ ] **Fast & Slow Pointers:** Cycle detection (`Floyd's`); `Linked List Cycle`, `Find Duplicate`, `Palindrome LL`
- [ ] **Sliding Window:** Fixed and variable windows; expand/shrink; `Longest Substring Without Repeat`, `Min Window Substring`
- [ ] **Kadane's Algorithm:** Max subarray; global vs. local max tracking; extension to max product subarray
- [ ] **Prefix Sum:** Cumulative sum array; range queries in O(1); `Subarray Sum Equals K`
- [ ] **Merge Intervals:** Sort by start time; merge overlapping; `Minimum Meeting Rooms`
- [ ] **In-Place Linked List Reversal:** Pointer re-linking without auxiliary memory; `Reverse Nodes in k-Group`
- [ ] **Monotonic Stack:** Next greater/smaller element; `Daily Temperatures`, `Remove K Digits`
- [ ] **Hash Map Tracking:** Single-pass look-back; complement search; frequency counting
- [ ] **Binary Search (Modified):** Search space reduction; rotated arrays; "search space of answers" (`Koko Bananas`)
- [ ] **Heaps / Top K:** `k`th largest; top-K frequent; median from data stream (two heaps)
- [ ] **Recursion & Backtracking:** State tree; candidate generation; prune; undo step; `Permutations`, `Combination Sum`
- [ ] **Tree Traversals:** DFS (pre/in/post-order); BFS (level-order); recursive vs. iterative
- [ ] **Graph Traversals:** BFS (shortest path in unweighted); DFS (connectivity, cycle detection, topological sort)
- [ ] **Shortest Path:** Dijkstra's (non-negative weights); Bellman-Ford (negative weights); Floyd-Warshall
- [ ] **Dynamic Programming:** Memoization (top-down) vs. tabulation (bottom-up); state definition; transition function
    - Patterns: 1D (`Climbing Stairs`, `House Robber`), 2D (`Unique Paths`), 0/1 Knapsack, LCS, LIS
- [ ] **Bit Manipulation:** AND, OR, XOR, left/right shifts; `Single Number`, `Count Bits`, `Power of Two`

2.3 Complexity & Problem-Solving Method `[CORE]`

- [ ] Time complexity: O(1), O(log n), O(n), O(n log n), $O(n^{2)}$, $^{O(2}n)$, O(n!) — know the differences concretely
- [ ] Space complexity: auxiliary space vs. total space; in-place algorithms; call stack depth in recursion
- [ ] Amortized analysis: ArrayList resizing, HashMap rehashing
- [ ] Problem-solving lifecycle: deconstruct → brute force → identify pattern → optimize → code review
- [ ] Recognize pattern keywords: "contiguous subarray" → Sliding Window/Kadane's; "shortest path" → BFS; "all combinations" → Backtracking

---

Phase 3: Databases — Relational & NoSQL

> ***Goal:*** *Go beyond CRUD. Understand the internals, failure behavior, and design trade-offs of the databases you actually use.*

3.1 PostgreSQL — Working Proficiency `[CORE]`

> *Reference: Trajectory (RDS + Flyway), Vigil (PostgreSQL 16), Phoenix (pgvector), Shard (monitoring), StudyLink (Supabase), Foundry (PostgreSQL) Core-CS Reference:* `D:\Work\Learning Engg\Core-CS-interview\CS\DBMS-BOSS-SHEET.pdf` *(Relational algebra, normalization 1NF–BCNF, ACID, transaction isolation anomalies, WAL)*

- [ ] PostgreSQL data types: numeric, text, `VARCHAR`, `TEXT`, `TIMESTAMP WITH TIME ZONE`, `UUID`, `JSONB`, `ARRAY`, custom `ENUM`
- [ ] Schema design: normalization levels; surrogate vs. natural keys; UUID vs. serial primary keys
  - → *Trajectory and StudyLink use UUID primary keys with PostgreSQL native ENUM types*
- [ ] Index types: B-tree (default), Hash, GiST, GIN, BRIN; partial and expression indexes
  - → *Phoenix and StudyLink add HNSW cosine indexes via pgvector for vector similarity*
- [ ] `EXPLAIN ANALYZE`: reading query plans; Sequential Scan vs. Index Scan vs. Bitmap Scan; cost estimation
- [ ] Transaction isolation levels and PostgreSQL's MVCC (Multi-Version Concurrency Control)
- [ ] Row-level locking: `SELECT FOR UPDATE`, `SELECT FOR SHARE`, `SKIP LOCKED`; deadlock detection
  - → *StudyLink, Conclave, Foundry use* `SELECT FOR UPDATE` *for state machine transitions*
- [ ] Connection pooling: PgBouncer, HikariCP; max connections, pool exhaustion, statement vs. session pooling
- [ ] Database migrations with Flyway: versioned scripts (`V1__init.sql`), `validate`, `repair`, rollback strategies
  - → *Trajectory runs* `V1__init_schema.sql` + `V2__add_missing_fields.sql` *with* `ddl-auto: validate`
- [ ] Alembic migrations (Python): `revision`, `upgrade`, `downgrade`, `autogenerate`
  - → *Vigil uses Alembic for PostgreSQL 16 schema management*
- [ ] Cascade operations: `ON DELETE CASCADE`, `ON DELETE SET NULL`; referential integrity
- [ ] Query optimization: `VACUUM`, `ANALYZE`, `pg_stat_user_tables`, slow query logging
- [ ] Backup strategies: `pg_dump`, `pg_restore`, WAL archiving concept
- [ ] pgvector extension: `VECTOR(n)` type, cosine distance (`<=>`), L2 distance (`<->`), inner product (`<#>`), HNSW vs. IVFFlat indexes
  - → *Phoenix uses* `all-MiniLM-L6-v2` *384-dim; StudyLink uses Gemini* `text-embedding-004` *768-dim*

3.2 PostgreSQL — Advanced Understanding [STRETCH]

- [ ] MVCC internals: tuple visibility, `xmin`/`xmax` columns, vacuum and dead tuples
- [ ] Write-ahead logging (WAL): durability mechanism, crash recovery, replication
- [ ] Partitioning: range, list, hash partitioning; partition pruning
- [ ] Logical and physical replication: streaming replication, replication slots
- [ ] `pg_stat_*` views: query statistics, lock contention monitoring
- [ ] Row-Level Security (RLS): policies, enabling/disabling per table

3.3 MySQL [OPTIONAL]

- [ ] InnoDB vs. MyISAM: transaction support, row-level locking, foreign keys
- [ ] MySQL-specific SQL: `SHOW PROCESSLIST`, `EXPLAIN FORMAT=JSON`, `AUTO_INCREMENT`
- [ ] Differences from PostgreSQL: `LIMIT x, y` syntax, `GROUP BY` rules, case sensitivity

3.4 Redis [CORE]

> *Reference: Foundry (Celery broker + channel layer), Trajectory (planned ElastiCache), Aegis targets*

- [ ] Redis as a data structure server: strings, hashes, lists, sets, sorted sets, bitmaps, HyperLogLog
- [ ] `GET`, `SET`, `DEL`, `EXPIRE`, `TTL`, `INCR`, `LPUSH`, `RPOP`, `ZADD`, `ZRANGEBYSCORE`
- [ ] Pub/Sub model; Redis Streams (`XADD`, `XREAD`, consumer groups)
- [ ] Persistence: RDB snapshots vs. AOF logging; `fsync` trade-offs
- [ ] Eviction policies: `noeviction`, `allkeys-lru`, `volatile-lru`, `allkeys-lfu`, etc.
  - → *Shard implements LRU and LFU from scratch; understand why Redis uses approximated LRU*
- [ ] Redis as a Celery broker: task queues, result backends, retry logic
  - → *Foundry uses Redis 7 as Celery task broker and Django Channels channel layer*
- [ ] Redis Cluster: hash slots (16384), cluster nodes, failover; when to use vs. single instance
- [ ] ElastiCache Redis on AWS: cluster mode, read replicas, encryption in transit/at rest, AUTH tokens
  - → *Trajectory's AWS provisioning runbook specifies ElastiCache Redis in private subnets; live deployment is unverified*
- [ ] Lua scripting: atomicity guarantee; `EVAL` command
- [ ] □ Verify: Explain the difference between `EXPIRE` and `PERSIST`; describe a cache stampede and how to prevent it

3.5 MongoDB [OPTIONAL]

> *Reference: IBM-PBEL training projects used MongoDB/Mongoose*

- [ ] Document model: collections, documents, embedded vs. referenced relationships
- [ ] CRUD: `insertOne/Many`, `find`, `updateOne/Many`, `deleteOne/Many`, `$set`, `$push`, `$pull`, `$inc`
- [ ] Aggregation pipeline: `$match`, `$group`, `$project`, `$lookup`, `$unwind`, `$sort`, `$limit`
- [ ] Indexes: single, compound, text, geospatial; explain plan
- [ ] Schema design with Mongoose: schemas, models, `populate`, validation
- [ ] ACID transactions in MongoDB 4.x+: session-based multi-document transactions

3.6 Database Concepts — Cross-Cutting `[CORE]`

- [ ] CAP theorem: Consistency, Availability, Partition Tolerance; trade-offs in practice
- [ ] ACID properties in depth: Atomicity, Consistency, Isolation, Durability
- [ ] BASE properties: Basically Available, Soft state, Eventually consistent; where NoSQL fits
- [ ] Consistency levels: strong, eventual, causal; how PostgreSQL MVCC achieves snapshot isolation
- [ ] Write-through vs. write-back vs. write-around caching semantics
  - → *Shard and Cairn simulate write-through and write-back pipelines explicitly*
- [ ] Cache invalidation strategies: TTL, event-driven invalidation, cache-aside, write-through
- [ ] Connection pool tuning: HikariCP (`maximumPoolSize`, `connectionTimeout`, `idleTimeout`)
- [ ] N+1 query problem: how it occurs with ORMs; solutions (JOIN fetching, batch loading)
- [ ] Optimistic locking: version columns, `@Version` in JPA; handling `OptimisticLockException`
- [ ] Pessimistic locking: when to use over optimistic; deadlock risk

---

Phase 4: Java Backend & Spring Boot Ecosystem

> ***Goal:*** *Systematic mastery of Spring Boot beyond "it works." Understand the container, the bean lifecycle, security model, and transaction semantics.*

4.1 Spring Core & Dependency Injection `[CORE]`

> *Reference: Cairn, Phoenix (gateway), Trajectory, Conclave, Smart Research Assistant*

- [ ] Inversion of Control (IoC) and Dependency Injection (DI): why they exist; constructor vs. setter vs. field injection
- [ ] Spring application context: `ApplicationContext` vs. `BeanFactory`; `AnnotationConfigApplicationContext`
- [ ] Bean lifecycle: instantiation → dependency injection → `@PostConstruct` → in use → `@PreDestroy`

- [ ] Bean scopes: `singleton` (default), `prototype`, `request`, `session`
- [ ] Annotations: `@Component`, `@Service`, `@Repository`, `@Controller`; stereotype model
- [ ] `@Autowired`, `@Qualifier`, `@Primary`; constructor injection best practice
- [ ] `@Configuration` and `@Bean` methods; `@ConditionalOnProperty`, `@Profile`
- [ ] Spring Boot autoconfiguration: `spring.factories`, `@EnableAutoConfiguration`, `@SpringBootApplication`
- [ ] `application.yml` / `application.properties`: externalized configuration; profiles (`spring.profiles.active`)
- [ ] `@ConfigurationProperties`: binding typed configuration classes
- [ ] □ Verify: Explain what happens from `main()` to your first HTTP request being served

4.2 Spring MVC & REST APIs [CORE]

- [ ] `DispatcherServlet`: front controller pattern; request lifecycle
- [ ] `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`
- [ ] `@PathVariable`, `@RequestParam`, `@RequestBody`, `@RequestHeader`, `@ResponseBody`
- [ ] `ResponseEntity<T>`: status codes, headers, body; when to use vs. returning the object directly
- [ ] `@ExceptionHandler`, `@ControllerAdvice`: global exception handling; standardized error responses
- [ ] Validation: `@Valid`, `@Validated`, Bean Validation annotations (`@NotNull`, `@Size`, `@Min`, `@Max`, `@Pattern`)
- [ ] REST API design principles: resource naming, HTTP methods, status codes, versioning, HATEOAS basics
- [ ] `@Async`: async method execution; `CompletableFuture` return type; thread pool configuration
  - → *Phoenix uses `@Async` for non-blocking HTTP calls from Spring Boot to FastAPI*
- [ ] `RestClient` and `WebClient`: synchronous vs. reactive HTTP clients; timeout configuration
  - → *Cairn uses `WebClient` for proxy routing; Phoenix raises timeout to 300s for Ollama cold start*
- [ ] Content negotiation, CORS configuration, versioning strategies

4.3 Spring Data JPA & Hibernate [CORE]

- [ ] ORM fundamentals: mapping Java objects to relational tables; entity lifecycle states (transient, managed, detached, removed)
- [ ] `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@Transient`
- [ ] Relationship mappings: `@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`; `mappedBy` and owner side
- [ ] Fetch strategies: `LAZY` vs. `EAGER`; the N+1 problem and solutions (`JOIN FETCH`, `@EntityGraph`, `@BatchSize`)
- [ ] `JpaRepository`, `CrudRepository`, `PagingAndSortingRepository`: derived query methods; `@Query` with JPQL/SQL
- [ ] `@Transactional`: propagation levels (`REQUIRED`, `REQUIRES_NEW`, `NESTED`); isolation levels; rollback rules
  - → *Understand why `@Transactional` on private methods does not work (Spring AOP proxy)*

- [ ] Transaction management: `TransactionSynchronizationManager`, programmatic transactions
- [ ] Hibernate-specific: `SessionFactory`, second-level cache, `@Cacheable`, `@NaturalId`
- [ ] `ddl-auto` settings: `create`, `update`, `validate`, `none`; why to use `validate` + Flyway in production
- [ ] Optimistic locking with `@Version`; handling `OptimisticLockException`
- [ ] JPA pessimistic locking: `@Lock(LockModeType.PESSIMISTIC_WRITE)`; `findWithLockById`
  - → *Conclave uses JPA pessimistic write locking for pipeline state transitions*
- [ ] Database connection pool: HikariCP defaults; tuning `maximumPoolSize`

4.4 Spring Security [CORE]

> *Reference: Trajectory (JWT + OAuth2), Phoenix (JWT multi-tenant), Conclave (JWT + STOMP auth), StudyLink*

- [ ] Security filter chain: request processing pipeline; `SecurityFilterChain` bean
- [ ] Authentication vs. Authorization: `AuthenticationManager`, `AuthenticationProvider`, `UserDetailsService`
- [ ] JWT authentication: `JwtAuthenticationFilter`; stateless session management (`SessionCreationPolicy.STATELESS`)
  - → *Trajectory, Conclave, Phoenix implement custom `JwtAuthenticationFilter` / `JwtAuthChannelInterceptor`*
- [ ] JWT structure: header.payload.signature; HMAC-SHA256 signing; token expiry and refresh rotation
- [ ] OAuth2 / OpenID Connect: authorization code flow, PKCE; `spring-security-oauth2-client`; Google/GitHub providers
  - → *Trajectory implements OAuth2 login alongside custom `MockOAuth2RedirectFilter` for local testing*
- [ ] RBAC: role-based access; `@PreAuthorize("hasRole('ADMIN')")`, `@Secured`, method security
- [ ] CSRF protection: when to disable (stateless REST), when to keep (session-based)
- [ ] CORS configuration in Spring Security: `CorsConfigurationSource`
- [ ] Password encoding: `BCryptPasswordEncoder`; why salting prevents rainbow table attacks
- [ ] WebSocket security: STOMP channel interceptor for WebSocket auth
  - → *Conclave uses `WebSocketAuthChannelInterceptor` for STOMP CONNECT authentication*
- [ ] Verify: Trace a JWT-authenticated request from HTTP header extraction through `SecurityContextHolder` population

4.5 Spring Boot — Advanced Topics [STRETCH]

- [ ] Spring Boot Actuator: health endpoints, metrics, info, custom health indicators
- [ ] Micrometer metrics: `Counter`, `Gauge`, `Timer`, `DistributionSummary`; Prometheus integration
  - → *Cairn exposes `cairn.cache` Micrometer metrics to Prometheus via Actuator*
- [ ] Spring WebSocket with STOMP: `@MessageMapping`, `SimpMessagingTemplate`, topic destinations
  - → *Conclave uses Spring WebSocket + STOMP for real-time LLM token streaming*
- [ ] Spring Scheduler: `@Scheduled`, `fixedRate`, `fixedDelay`, `cron` expressions; `@EnableScheduling`

- → *Trajectory uses `@Scheduled(cron="0 0 1 * * ?")` for daily ghost detection*
- [ ] Event-driven within Spring: `ApplicationEvent`, `@EventListener`, `ApplicationEventPublisher`
- [ ] Spring Batch: job, step, reader, processor, writer; use cases vs. Celery/queues
- [ ] Custom starters and autoconfiguration: `META-INF/spring.factories`
- [ ] Testing: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `MockMvc`, `@MockBean`, `TestRestTemplate`

Phase 5: Python Backend Frameworks

> ***Goal:*** *Learn FastAPI and Django beyond "hello world." Understand ASGI, sync vs. async, middleware, and ORM behavior.*

5.1 FastAPI [CORE]

> *Reference: Vigil (REST API + Uvicorn), Phoenix (AI engine), Shard (REST API), Warden targets*

- [ ] ASGI vs. WSGI: asynchronous server gateway interface; why FastAPI uses ASGI
- [ ] `FastAPI()` application, `APIRouter`, path operations (`@app.get`, `@app.post`, etc.)
- [ ] Pydantic v2 for request/response validation: `BaseModel`, `Field`, validators, `model_validator`
  - → *Vigil uses Pydantic v2 discriminated unions for evaluation assertion models*
- [ ] Dependency injection system: `Depends()`; scoped dependencies; `yield` dependencies for resource cleanup
- [ ] Path parameters, query parameters, request bodies, headers; response models and `response_model`
- [ ] `HTTPException`; custom exception handlers with `@app.exception_handler`
- [ ] Background tasks: `BackgroundTasks`; difference from async endpoints
- [ ] `async def` endpoints vs. `def` (sync in thread pool); when each is appropriate
- [ ] Uvicorn: ASGI server; `--workers`, `--reload`, graceful shutdown
- [ ] Static file mounting: `StaticFiles`, SPA serving
  - → *Vigil mounts a vanilla JS/CSS SPA directly from FastAPI `StaticFiles`*
- [ ] Testing FastAPI: `TestClient` (synchronous), `AsyncClient` with `httpx`
- [ ] OpenAPI/Swagger auto-generation; `tags`, `summary`, `description`, `response_description`

5.2 Django & Django REST Framework [CORE]

> *Reference: Shard (DRF cache API), Foundry (Django + Channels + Celery), StudyLink (Django 5.1)*

- [ ] Django project structure: `settings.py`, `urls.py`, `wsgi.py`/`asgi.py`, apps
- [ ] Django ORM: models, migrations, `QuerySet` API, `select_related`, `prefetch_related`
- [ ] Django admin, custom managers, model signals (`post_save`, `pre_delete`)
- [ ] `AbstractBaseUser` and custom user models: why to create early; `USERNAME_FIELD`, `REQUIRED_FIELDS`
  - → *StudyLink, Foundry use custom `AbstractBaseUser` with email authentication and UUID PKs*
- [ ] Django REST Framework: `APIView`, `ViewSet`, `ModelViewSet`; serializers, `HyperlinkedModelSerializer`
  - → *Shard uses class-based DRF views for all cache API endpoints*
- [ ] DRF authentication: `TokenAuthentication`, `SessionAuthentication`, `djangorestframework-simplejwt`
- [ ] DRF permissions: `IsAuthenticated`, `IsAdminUser`, `AllowAny`; custom permission classes
- [ ] DRF throttling: `AnonRateThrottle`, `UserRateThrottle`, custom throttle classes; Redis-backed throttling
  - → *Foundry uses `TierBasedRateThrottle` with Redis sliding-window rate limiting*
- [ ] Pagination: `PageNumberPagination`, `CursorPagination`; custom `StandardResultsSetPagination`
- [ ] `@transaction.atomic()` and `transaction.on_commit()` hooks
  - → *StudyLink uses `transaction.on_commit()` for post-transaction notification dispatch*
- [ ] Django Channels (ASGI): `CHANNEL_LAYERS`, consumers, `AsyncWebsocketConsumer`, WebSocket routing
  - → *Foundry uses Django Channels + Redis channel layer for real-time WebSocket streaming*
- [ ] Celery with Django: task definition (`@shared_task`), `.delay()`, `.apply_async()`, Redis broker, result backend
  - → *Foundry dispatches LangGraph execution to Celery workers*
- [ ] Daphne ASGI server: serving Django Channels over HTTP and WebSocket
- [ ] Django settings management: `django-environ`, `python-dotenv`, environment-based settings

5.3 Python Web — Advanced `[STRETCH]`

- [ ] Pydantic settings: `BaseSettings` for environment variable binding; secrets support
- [ ] `httpx` async HTTP client: `AsyncClient`, connection pooling, timeouts, `respx` for mocking
  - → *Shard uses pooled `httpx.Client` for transparent cluster proxy routing*
- [ ] SQLAlchemy 2.0: `DeclarativeBase`, `Session`, `async_session`, `select()`, relationship loading
  - → *Vigil and Phoenix use SQLAlchemy 2.0 with Alembic for schema management*
- [ ] Tenacity retry library: `@retry`, `stop_after_attempt`, `wait_exponential`, `retry_if_exception_type`
  - → *Foundry uses Tenacity for Gemini API exponential retry pipelines*
- [ ] Poetry: dependency groups, virtual environments, `pyproject.toml`, `poetry.lock` reproducibility

---

Phase 6: Backend Architecture, API Design & Systems Concepts

> ***Goal:*** *Move from "makes it work" to "makes it work reliably at scale." These are the concepts interviewers probe deeply.*

6.1 RESTful API Design `[CORE]`

- [ ] REST constraints: statelessness, uniform interface, client-server, cacheability, layered system
- [ ] Resource naming: nouns not verbs; plural resources; nested resources; flat vs. nested trade-offs
- [ ] HTTP methods semantics: `GET` (safe, idempotent), `POST` (not idempotent), `PUT` (idempotent), `PATCH`, `DELETE`
- [ ] HTTP status codes: 2xx, 3xx, 4xx, 5xx; know the meaning of 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500, 503
- [ ] API versioning strategies: URL path (`/v1/`), header, query parameter; trade-offs
- [ ] Idempotency: designing idempotent endpoints; idempotency keys for POST operations
- [ ] Rate limiting: token bucket, leaky bucket, sliding window; `429 Too Many Requests`; `Retry-After` header
- [ ] Pagination: cursor-based vs. offset-based; cursor stability across writes
- [ ] HATEOAS basics: hypermedia links in responses; Spring HATEOAS
- [ ] OpenAPI / Swagger specification: documenting APIs; contract-first vs. code-first
- [ ] Error response standardization: RFC 7807 Problem Details; consistent error schema
- [ ] Verify: Design a versioned REST API for a resource with sub-resources; justify each status code choice

6.2 Authentication & Authorization `[CORE]`

- [ ] Session-based authentication: server-side session store; scaling issues
- [ ] JWT: stateless authentication; token structure; signing algorithms (HS256 vs. RS256); refresh token rotation
  - → *Trajectory implements 7-day refresh token rotation with in-flight refresh mutex in frontend*
- [ ] OAuth 2.0 flows: authorization code (+ PKCE), implicit (deprecated), client credentials, device code
- [ ] OpenID Connect: ID token vs. access token; claims; JWKS endpoint
- [ ] API keys: when appropriate vs. OAuth; key management and rotation
- [ ] RBAC (Role-Based Access Control): roles, permissions, policy enforcement at API layer
- [ ] Token storage: `localStorage` vs. `HttpOnly` cookies; XSS and CSRF trade-offs
  - → *StudyLink stores access tokens in React memory, refresh tokens in `HttpOnly SameSite=Lax` cookies*
- [ ] Multi-tenancy: tenant isolation strategies; row-level security; schema-per-tenant

6.3 HTTP, Networking & Transport [CORE]

> *Core-CS Reference: `D:\Work\Learning Engg\Core-CS-interview\CS\Computer-Network-Boss-Sheet.pdf` (TCP/IP stack, OSI layers, socket lifecycle, flow control, DNS resolution, TLS handshake)*

- [ ] HTTP/1.1 vs. HTTP/2 vs. HTTP/3: multiplexing, header compression, QUIC
- [ ] TCP: three-way handshake, connection establishment, FIN/RST termination; TCP vs. UDP
- [ ] TLS/SSL: handshake, certificate validation, mutual TLS; `HTTPS`; certificate chain
- [ ] DNS: resolution process, A records, CNAME, TTL; round-robin DNS
- [ ] WebSockets: upgrade handshake, full-duplex communication, framing; STOMP protocol over WebSocket
  - → *Foundry, Conclave use WebSocket + STOMP for LLM token streaming; understand keep-alive and backpressure*
- [ ] Long polling vs. SSE vs. WebSocket: when to use each
- [ ] HTTP caching: `Cache-Control`, `ETag`, `Last-Modified`, `If-None-Match`, `If-Modified-Since`
- [ ] Load balancers: L4 vs. L7; round-robin, least connections, consistent hashing; sticky sessions
- [ ] CDN: edge caching, cache invalidation, origin pull; when to use
- [ ] Proxies vs. reverse proxies: Nginx as reverse proxy; upstream blocks, SSL termination
- [ ] Content Security Policy, CORS, Same-Origin Policy
- [ ] Verify: Trace an HTTPS request from DNS resolution through TLS handshake to HTTP response

6.4 Distributed Systems Fundamentals [CORE]

- [ ] Fallacies of distributed computing (the 8 fallacies): network is reliable, latency is zero, bandwidth is infinite, etc.
- [ ] Consistency models: strong, eventual, causal, read-your-writes, monotonic reads
- [ ] Distributed transactions: two-phase commit (2PC); Saga pattern (choreography vs. orchestration)
- [ ] Event-driven architecture: event sourcing, CQRS; message brokers
- [ ] Idempotency in distributed systems: at-least-once vs. at-most-once vs. exactly-once delivery
- [ ] Service discovery: client-side vs. server-side; Eureka, Consul (conceptual)
- [ ] Circuit breaker pattern: Closed → Open → Half-Open; Resilience4j
- [ ] Bulkhead pattern: resource isolation; thread pool isolation
- [ ] Retries with exponential backoff and jitter: avoiding thundering herd
  - → *Foundry uses Tenacity exponential retry for Gemini API calls*

6.5 Caching Architecture [CORE]

> *Reference: Shard, Cairn, Redis in Foundry, ElastiCache in Trajectory infrastructure runbook*

- [ ] Cache-aside (lazy loading): application loads cache on miss
- [ ] Write-through: every write goes to cache and DB simultaneously
  - → *Shard and Cairn implement simulated write-through vs. write-back pipelines*
- [ ] Write-back (write-behind): write to cache; async flush to DB; durability risk
- [ ] Read-through: cache reads through to DB automatically
- [ ] Cache eviction policies: LRU, LFU, FIFO, MRU, random; when each is appropriate
  - → *Shard implements O(1) LRU (sentinel doubly-linked list) and LFU (frequency-bucket OrderedDict)*
  - → *Cairn implements concurrent O(1) LRU in Java with* `ReentrantReadWriteLock`
- [ ] Cache invalidation: the hard problem; TTL, event-driven invalidation, cache busting
- [ ] Consistent hashing: virtual nodes, Murmur3-32, ring wraparound; minimal key migration
  - → *Shard and Cairn both implement Murmur3-32 consistent hash rings with 150 virtual nodes*
- [ ] Cache stampede / thundering herd: causes and mitigation (probabilistic early expiration, mutex, background refresh)
- [ ] Distributed cache (Redis Cluster): hash slots, node assignment, failover, client-side routing
- [ ] Cache hit rate, miss rate, eviction rate: metrics and impact on system design

6.6 Message Queues & Event Streaming [CORE]

> *Reference: Celery (Foundry), Redis Pub/Sub, Django Channels*

- [ ] Message queue fundamentals: producers, consumers, brokers, topics, queues
- [ ] Message semantics: at-most-once, at-least-once, exactly-once delivery guarantees
- [ ] Celery architecture: worker processes, beat scheduler, broker (Redis/RabbitMQ), result backend
  - → *Foundry uses Celery 5.6 with Redis broker for async LangGraph task dispatch*
- [ ] Task retries, timeouts, priority queues, task routing in Celery
- [ ] Kafka concepts [STRETCH]: topics, partitions, offsets, consumer groups, log compaction, retention
- [ ] RabbitMQ concepts [STRETCH]: exchanges (direct, fanout, topic, headers), queues, bindings, acknowledgments
- [ ] Redis Pub/Sub: channel subscription, message publishing; no persistence guarantee
- [ ] When to use queues vs. direct HTTP calls vs. WebSockets: latency, decoupling, reliability trade-offs

---

Phase 7: Frontend Engineering

7.1 HTML & CSS Foundations [CORE]

- [ ] HTML5 semantic elements: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`
- [ ] Forms: `<form>`, `<input>` types, `<label>`, `<select>`, `<textarea>`, native validation attributes
- [ ] Accessibility (a11y): ARIA roles, `aria-label`, `aria-describedby`, semantic HTML for screen readers; keyboard navigation

- [ ] CSS box model: `content`, `padding`, `border`, `margin`; `box-sizing: border-box`
- [ ] Flexbox: `flex-direction`, `justify-content`, `align-items`, `flex-wrap`, `flex-grow/shrink/basis`
- [ ] CSS Grid: `grid-template-columns/rows`, `gap`, `grid-area`, `auto-fill`, `minmax`
- [ ] CSS custom properties (variables): `--color-primary`; dynamic theming
  - → *Foundry, Vigil, and Conclave frontends use CSS custom properties for design tokens*
- [ ] Media queries and responsive design: `min-width`, `max-width`, mobile-first approach
- [ ] CSS animations and transitions: `transition`, `@keyframes`, `animation`; `will-change` for GPU compositing
- [ ] CSS specificity, inheritance, cascade order
- [ ] Tailwind CSS `[OPTIONAL]`: utility-first approach; `tailwind.config.js`; responsive variants; dark mode
  - → *Trajectory and StudyLink use Tailwind CSS; Check-Mate Analyse uses Tailwind*

7.2 React `[CORE]`

> *Reference: Phoenix (React 19), Foundry (React 19 + Zustand), Trajectory (React 19 + TypeScript), Conclave (React 19), StudyLink (React 18), Check-Mate Analyse (React 19)*

- [ ] JSX: transpilation to `React.createElement`; why `.jsx`/`.tsx` is needed
- [ ] Components: functional components; props; children; key prop (reconciliation)
- [ ] `useState`: local state; functional updates; state initialization
- [ ] `useEffect`: side effects; dependency array; cleanup functions; common pitfalls (infinite loops)
- [ ] `useRef`: mutable container; DOM refs; accessing ref in event handlers without re-renders
  - → *Check-Mate Analyse uses `useRef` to sync WebWorker callbacks without triggering re-renders*
- [ ] `useCallback`, `useMemo`: memoization; when they help and when they add overhead
- [ ] `useReducer`: complex state machines; action-dispatch pattern
  - → *Check-Mate Analyse uses `useReducer` as the single state store for the chess workbench*
- [ ] `useContext`: context API; context provider; consuming context; performance pitfalls
- [ ] Custom hooks: extracting reusable stateful logic; naming convention `use*`
- [ ] React 19 features: `use()` hook, Actions, form integration, compiler (concept)
- [ ] React rendering model: reconciliation; virtual DOM; fiber architecture concepts
- [ ] Lists and keys: why keys must be stable and unique; reconciliation behavior
- [ ] Event handling: synthetic events; event delegation; `stopPropagation`, `preventDefault`
- [ ] React Router: `BrowserRouter`, `Routes`, `Route`, `Link`, `useNavigate`, `useParams`, `useLocation`, protected routes
- [ ] Error boundaries: `componentDidCatch`, `getDerivedStateFromError`; `react-error-boundary`
- [ ] `React.memo`, `React.lazy`, `Suspense`: code splitting and lazy loading

7.3 State Management `[CORE]`

- [ ] Local state vs. global state vs. server state: when each applies

- [ ] Zustand: `create()`, selectors, slice pattern, `devtools` middleware, `immer` middleware
  - → *Foundry, Trajectory, Conclave, StudyLink all use Zustand; Trajectory implements in-flight refresh mutex in api layer*
- [ ] TanStack Query (React Query): `useQuery`, `useMutation`, `queryKey`, `staleTime`, `gcTime`, cache invalidation, optimistic updates
  - → *Trajectory uses TanStack Query for all server state; manual `invalidateQueries` after mutations*
- [ ] Context API: when sufficient vs. when to reach for Zustand; context value stability
- [ ] Redux `[OPTIONAL]`: store, reducers, actions, `useSelector`, `useDispatch`, Redux Toolkit (`createSlice`, `createAsyncThunk`)

7.4 React Ecosystem & Tooling `[CORE]`

- [ ] Vite: dev server, HMR, production build with Rollup; `vite.config.ts`; environment variables (`VITE_*`)
- [ ] TypeScript with React: `FC<Props>`, `ReactNode`, `ReactElement`, typing hooks, generic components
- [ ] Axios: interceptors, base URL, token injection, `401` retry with subscriber queue pattern
  - → *Trajectory and StudyLink implement an in-flight refresh mutex with subscriber queue to prevent parallel refresh races*
- [ ] Form handling: controlled vs. uncontrolled inputs; React Hook Form (optional); validation with Zod
- [ ] Testing with Vitest: `describe`, `it`, `expect`, `vi.mock`; `jsdom` environment; `@testing-library/react`
  - → *Trajectory uses Vitest to test Zustand stores; includes `window.matchMedia` mock setup*
- [ ] Playwright basics: browser automation; `page.goto`, `page.fill`, `page.click`, `expect(locator).toBeVisible()`
  - → *Foundry uses Playwright for E2E tests; Aegis is a full Playwright framework targeting Trajectory*

7.5 Advanced Frontend `[STRETCH]`

- [ ] Performance optimization: `React.memo`, virtualization (`react-window`/`react-virtual`), code splitting, lazy loading
- [ ] Rendering strategies: CSR, SSR, SSG, ISR; when Next.js adds value over Vite
- [ ] Web Vitals: LCP, CLS, FID/INP; measuring with Lighthouse and DevTools
- [ ] Service Workers: offline support, push notifications; PWA concepts
- [ ] WebAssembly from React: loading WASM modules; `postMessage` bridge to Web Workers
  - → *Check-Mate Analyse loads Stockfish WASM inside a Web Worker; requires COOP/COEP headers*
- [ ] `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy`: required for `SharedArrayBuffer` and WASM threading
- [ ] Recharts / Chart.js: declarative chart components; custom tooltips; click-to-navigate interactions
  - → *Check-Mate Analyse uses Recharts for evaluation curve; Trajectory uses Recharts for pipeline analytics*

Phase 8: DevOps, Containers & Cloud Deployment

8.1 Docker [CORE]

> *Reference: Vigil (sandbox containers with Docker SDK), Shard (multi-node Docker Compose), Trajectory (multi-stage Dockerfile), Aegis/Pulse/Warden (Docker-based test environments)*

- [ ] Docker architecture: daemon, client, image, container, registry; image layers and caching
- [ ] `Dockerfile`: `FROM`, `RUN`, `COPY`, `WORKDIR`, `EXPOSE`, `CMD`, `ENTRYPOINT`; multi-stage builds
  - → *Trajectory uses multi-stage Eclipse Temurin 21 JRE Alpine for lightweight production images*
- [ ] Image optimization: minimizing layers, using Alpine base images, `.dockerignore`
- [ ] Container lifecycle: `docker run`, `docker stop`, `docker rm`, `docker ps`, `docker logs`, `docker exec`
- [ ] Container resource limits: `--memory`, `--cpus`; cgroup enforcement
  - → *Vigil caps sandbox containers at 512MB RAM and 0.5 CPU; WSL2 memory overcommit nuances*
- [ ] Security hardening: non-root users, read-only root filesystems, `--cap-drop=ALL`, `--security-opt=no-new-privileges`
  - → *Vigil: non-root UID 1000, read-only root, all caps dropped, network disabled — defense-in-depth*
- [ ] Volumes and mounts: named volumes, bind mounts, tmpfs; data persistence strategy
- [ ] Networks: bridge, host, none; container-to-container communication by service name
- [ ] `docker-compose.yml`: services, depends_on, healthchecks, environment variables, port mapping
  - → *Shard runs 3-node distributed cache cluster via Docker Compose; Foundry: Django + Celery + Redis*
- [ ] Docker SDK for Python: `docker-py`; `client.containers.run()`, `container.exec_run()`, `container.kill()`
  - → *Vigil uses Docker SDK to provision, manage, and destroy ephemeral sandbox containers programmatically*
- [ ] Container cleanup guarantees: `try/finally` context managers, signal handlers, registry cleanup
  - → *Vigil's 3-tier cleanup: `try/finally` + `ActiveContainerRegistry` + `signal.signal(SIGINT)` + Docker labels*
- [ ] Docker networking debugging: `docker network inspect`, DNS resolution, bridge networking
- [ ] □ Verify: Build a multi-stage Dockerfile; explain the layer cache invalidation; identify security improvements

8.2 CI/CD & GitHub Actions [CORE]

> *Reference: Trajectory (CI workflows), Aegis (CI pipeline), Pulse (validation workflow)*

- [ ] CI/CD concepts: continuous integration, continuous delivery, continuous deployment; pipeline stages
- [ ] GitHub Actions: workflow YAML structure; `on` triggers (push, PR, schedule, manual); `jobs`, `steps`, `runs-on`
- [ ] Environment variables and secrets in GitHub Actions: `${{ secrets.* }}`, `${{ env.* }}`
- [ ] Self-hosted runners: setup, label assignment; security considerations
  - → *Trajectory runbook outlines self-hosted EC2 runner setup for container rebuilding*
- [ ] Docker in GitHub Actions: building, tagging, and pushing to ECR; `docker/build-push-action`
- [ ] Deployment strategies: rolling updates, blue-green, canary; zero-downtime deployment
- [ ] Caching in CI: dependency caching (`actions/cache`); build cache
- [ ] Branch protection rules: required status checks; PR review requirements
- [ ] Workflow artifacts: uploading and downloading test results, reports
- [ ] Verify: Write a GitHub Actions workflow that builds, tests, and deploys a container

8.3 AWS Cloud [CORE]

> *Reference: Trajectory infrastructure runbook (documented AWS architecture: EC2/ECS, S3, RDS, ElastiCache in `aws-provisioning-guide.md`)*

- [ ] IAM: users, groups, roles, policies; principle of least privilege; instance profiles for EC2
- [ ] EC2: instance types, AMIs, key pairs, security groups, elastic IPs, user data scripts
  - → *Trajectory infrastructure runbook documents EC2 provisioning with Nginx reverse proxy*
- [ ] S3: buckets, objects, presigned URLs, bucket policies, ACLs; S3 as object storage
  - → *Trajectory's application code integrates with S3 for versioned resume and company-document storage; production usage is unverified*
- [ ] RDS: managed PostgreSQL, Multi-AZ, read replicas, automated backups, parameter groups
  - → *Trajectory infrastructure runbook specifies RDS PostgreSQL 16 with Multi-AZ and Secrets Manager*
- [ ] VPC: subnets (public vs. private), security groups vs. NACLs, internet gateway, NAT gateway, routing tables
  - → *Trajectory infrastructure runbook designs ALB in public subnets, ECS + RDS + ElastiCache in private subnets*
- [ ] ECS Fargate: task definitions, services, clusters, target groups; container-based compute without EC2 management
  - → *Trajectory infrastructure runbook designs ECS Fargate cluster with ALB forwarding*
- [ ] ECR: container image registry; `aws ecr get-login-password`, push/pull workflow
- [ ] ElastiCache Redis: cluster configuration, node types, encryption in transit, AUTH token
  - → *Trajectory infrastructure runbook specifies ElastiCache Redis 7 in private subnet; live deployment is unverified*
- [ ] Application Load Balancer: listeners, rules, target groups; health checks; HTTPS termination
- [ ] AWS Secrets Manager: secret storage; referencing in ECS task definitions; rotation
- [ ] CloudWatch: logs, metrics, alarms; log groups and streams

8.4 AWS — Advanced & Deployment Patterns [STRETCH]

- [ ] ECS service auto scaling: target tracking; step scaling; scheduled scaling
- [ ] Amazon S3 presigned URLs: generating, expiry, secure access patterns
- [ ] Lambda: function-as-a-service; event sources (API Gateway, S3, SQS); cold starts; memory/timeout tuning
- [ ] CloudFront: CDN behavior; cache behaviors; origin access control for S3
- [ ] Route 53: hosted zones, record types (A, CNAME, ALIAS), health checks, routing policies
- [ ] Systems Manager Parameter Store: secure parameter management without Secrets Manager cost
- [ ] Cost optimization: Reserved Instances, Spot Instances; S3 storage classes; RDS reserved instances

8.5 Monitoring, Logging & Observability [STRETCH]

> *Reference: Shard/Cairn (Prometheus + Grafana), Pulse (InfluxDB + Grafana), Vigil (P50/P90 telemetry)*

- [ ] Observability pillars: metrics, logs, traces; the difference between monitoring and observability
- [ ] Prometheus: metrics exposition (`/metrics`), `counter`, `gauge`, `histogram`, `summary`; scrape interval
  - → *Shard exposes Prometheus metrics; Cairn uses Micrometer/Actuator for Prometheus integration*
- [ ] Grafana: data source configuration; dashboard panels; alert rules; provisioning via JSON
  - → *Shard and Cairn include `grafana-dashboard.json`; Pulse provisions InfluxDB + Grafana for k6 metrics*
- [ ] Application logging: structured logging (JSON); log levels; correlation IDs; log aggregation (ELK, CloudWatch Logs)
- [ ] Distributed tracing: trace IDs, span IDs, Jaeger, Zipkin; B3 header propagation (concept)
- [ ] Percentile metrics: P50, P90, P95, P99; why averages mislead; tail latency
  - → *Vigil implements pure-Python P50/P90 linear interpolation (Excel PERCENTILE.INC standard)*
  - → *Shard exposes P50/P95/P99 rolling latency; Pulse tracks p95/p99 with k6*
- [ ] Alerting: SLO, SLI, SLA definitions; error budget concept; alert fatigue
- [ ] Health checks: liveness vs. readiness probes; `pg_isready`; Docker `HEALTHCHECK`
- [ ] InfluxDB + k6: time-series metrics storage; `docker-compose` Grafana+InfluxDB+k6 stack
  - → *Pulse uses InfluxDB as the k6 metrics backend with pre-provisioned Grafana dashboards*
- [ ] Nginx basics: `upstream`, `proxy_pass`, `gzip`, rate limiting, SSL termination; `nginx.conf` structure

---

Phase 9: Low-Level Design (LLD)

> ***Career relevance:*** *STRETCH for most engineering roles; CORE for product-company interviews where LLD rounds are common. Reference Cheatsheet:* `D:\Work\Learning Engg\Core-CS-interview\SD\LLD_Design Pattern Boss sheet.pdf`

9.1 SOLID Principles [STRETCH]

- [ ] **S** — Single Responsibility Principle: one reason to change; how to identify violations
- [ ] **O** — Open/Closed Principle: open for extension, closed for modification; interfaces and abstract classes
- [ ] **L** — Liskov Substitution Principle: subtypes must be substitutable; behavioral subtyping
- [ ] **I** — Interface Segregation Principle: fat interfaces; role interfaces; avoid forcing unused methods
- [ ] **D** — Dependency Inversion Principle: depend on abstractions; constructor injection
- [ ] □ Verify: Identify an SOLID violation in your own project code and refactor it

9.2 Design Patterns [STRETCH]

- [ ] **Creational:** Factory Method, Abstract Factory, Builder, Singleton (and its thread-safety concerns), Prototype
- [ ] **Structural:** Adapter, Decorator, Proxy, Facade, Bridge, Composite, Flyweight
- [ ] **Behavioral:** Strategy, Observer, Command, Iterator, State, Chain of Responsibility, Mediator, Template Method
- [ ] Strategy Pattern in depth: pluggable algorithms; `EvictionPolicy` interface
  - → *Shard uses Strategy Pattern for LRU/LFU eviction; Vigil uses pluggable* `BaseAgentAdapter`
- [ ] Observer Pattern: event publishing; `ApplicationEventPublisher` in Spring
- [ ] Adapter Pattern: bridging incompatible interfaces
  - → *Conclave uses* `ModelAdapter` *(LlamaAdapter, MistralAdapter, GemmaAdapter) to normalize LLM APIs*
- [ ] Template Method: base class defines skeleton; subclasses fill in steps

9.3 UML & Object Modeling [STRETCH]

- [ ] Class diagrams: classes, relationships (association, aggregation, composition, inheritance, dependency)
- [ ] Sequence diagrams: object interactions over time; message passing
- [ ] State diagrams: states, transitions, guards; mapping to code state machines

9.4 Classic LLD Problems [STRETCH]

- [ ] Parking Lot: multi-level, vehicle types, ticketing, payment
- [ ] BookMyShow / Movie Ticket Booking: seats, shows, bookings, concurrent reservation
- [ ] Splitwise: groups, expenses, simplify debts algorithm
- [ ] Elevator System: multiple elevators, scheduling algorithm (LOOK/SCAN/FCFS)
- [ ] Chess: board, pieces, moves, check/checkmate detection
- [ ] Library Management System: books, members, reservations, fines
- [ ] In-memory cache (LRU/LFU from scratch): without using Java `LinkedHashMap`
  - → *Shard and Cairn are real implementations; review their eviction code before mock interviews*

Phase 10: High-Level Design (HLD) & System Design

> ***Career relevance:*** *CORE for senior/experienced-engineer interviews; essential foundational knowledge even for junior roles. Reference Cheatsheet:* `D:\Work\Learning Engg\Core-CS-interview\SD\HLD system design Boss sheet.pdf`

10.1 Scalability Fundamentals [CORE]

- [ ] Vertical vs. horizontal scaling: when to scale up vs. out
- [ ] Stateless vs. stateful services: why stateless services scale horizontally; session affinity trade-offs
- [ ] Replication: master-slave (leader-follower), multi-master; read replicas; replication lag
- [ ] Sharding: horizontal partitioning; range-based vs. hash-based sharding; cross-shard queries
- [ ] Consistent hashing: ring topology, virtual nodes, minimal key migration
  - → *Shard and Cairn implement consistent hashing from scratch; review this before any HLD interview*
- [ ] Load balancing: round-robin, least connections, IP hash; L4 vs. L7 load balancers
- [ ] Caching layers: CDN → API gateway cache → application cache → database cache
- [ ] Database read/write split: routing reads to replicas; write to primary; replication lag handling
- [ ] Geographic distribution: multi-region deployments, latency-based routing, data sovereignty

10.2 Reliability & Fault Tolerance [CORE]

- [ ] Availability: SLA percentages (99.9% = 8.76 hrs/year downtime); failure modes
- [ ] Single points of failure (SPOF): identifying and eliminating them
- [ ] Replication for fault tolerance: N replicas, quorum-based reads/writes
- [ ] Circuit breaker: preventing cascade failures; Resilience4j; states and thresholds
- [ ] Timeouts and retries: exponential backoff, jitter; retry budgets
- [ ] Graceful degradation: returning stale data vs. error; partial functionality
- [ ] Bulkhead: isolated thread pools; resource limits per service

10.3 Message Queues & Event-Driven Systems [CORE]

- [ ] Why queues: decoupling, buffering, backpressure, async processing
- [ ] Kafka architecture: broker, topic, partition, offset, consumer group; exactly-once semantics
- [ ] RabbitMQ: exchange types, binding, acknowledgment, DLQ (Dead Letter Queue)
- [ ] Outbox Pattern: reliable event publishing from a transaction; CDC (Change Data Capture)
- [ ] Event sourcing: events as the source of truth; rebuilding state from event log
- [ ] CQRS: separate read and write models; eventual consistency between models

10.4 API Gateway & Microservices [CORE]

- [ ] API Gateway: single entry point; rate limiting, authentication, request routing, SSL termination
- [ ] Microservices: bounded contexts, service contracts, independent deployability

- [ ] Service mesh (concept): Istio, Linkerd; sidecar proxies; mutual TLS between services
- [ ] Service discovery: Consul, Eureka; health check registration
- [ ] Inter-service communication: REST, gRPC, async message passing; choosing the right transport

10.5 Classic HLD Problems [CORE]

- [ ] Design YouTube: video upload, encoding pipeline, streaming, recommendations, view count
- [ ] Design WhatsApp: message delivery, encryption, offline queuing, group messaging
- [ ] Design Uber: real-time location matching, surge pricing, trip state machine
- [ ] Design Twitter/X: timeline generation (fan-out on write vs. read), trending topics
- [ ] Design Netflix: video storage, adaptive bitrate streaming, CDN strategy
- [ ] Design a URL shortener: hash generation, collision handling, redirect, analytics
- [ ] Design a distributed cache (Redis-like): eviction, clustering, replication, consistency
  - → *Shard and Cairn provide concrete implementation context for this problem*
- [ ] Design an evaluation harness / sandboxed execution system
  - → *Vigil provides direct implementation reference: ephemeral Docker sandboxes, assertion scoring, telemetry*

10.6 CAP Theorem & Distributed Consistency [CORE]

- [ ] CAP theorem: cannot have all three simultaneously; which two to choose and when
- [ ] Consistency vs. availability trade-offs in practice: Zookeeper (CP) vs. DynamoDB (AP)
- [ ] PACELC extension: latency vs. consistency during normal operation (even without partition)
- [ ] Eventual consistency: read-your-writes, monotonic reads, causal consistency
- [ ] Distributed consensus: Raft, Paxos (conceptual); etcd, ZooKeeper

---

Phase 11: Generative AI & AI Engineering

> ***Career relevance:*** *CORE if targeting AI Engineer roles; STRETCH for Java/Python backend; OPTIONAL for pure frontend/QA tracks.*

11.1 ML & LLM Foundations [CORE for AI track]

- [ ] Neural network basics: layers, neurons, weights, biases, activation functions, backpropagation concept
- [ ] Transformer architecture: self-attention mechanism, positional encoding, encoder-decoder vs. decoder-only
- [ ] LLM fundamentals: pre-training, fine-tuning, RLHF; context window; tokenization
- [ ] Temperature, top-p, top-k: how they affect generation randomness
- [ ] Hallucination: causes (training data gaps, context mismatch); mitigation strategies
- [ ] Token limits and context management: chunking strategies; context compression

- [ ] Model selection: GPT-4, Claude, Gemini, Llama 3, Mistral, Gemma; trade-offs (capability, cost, privacy, speed)
- [ ] Embeddings: dense vector representations; cosine similarity; semantic search
- [ ] Vector databases: pgvector, FAISS, Pinecone, Qdrant; ANN (Approximate Nearest Neighbor) algorithms
  - → *Phoenix uses pgvector HNSW index; StudyLink uses pgvector with Gemini embeddings*

11.2 Prompt Engineering `[CORE for AI track]`

- [ ] System prompts vs. user prompts vs. assistant messages; few-shot prompting
- [ ] Chain-of-thought (CoT) prompting; structured output prompting with JSON schemas
- [ ] Prompt injection: attacks and mitigation
- [ ] Structured output generation: JSON mode, schema-constrained generation
  - → *Trajectory uses Spring AI with Java record schema binding for structured LLM extraction*
  - → *Foundry uses Gemini structured generation via Pydantic schema*
- [ ] Prompt template management: parameterized templates; versioning prompts
- [ ] □ Verify: Design a prompt that reliably produces structured JSON output and write a unit test for it

11.3 RAG — Retrieval-Augmented Generation `[CORE for AI track]`

> *Reference: Phoenix (hybrid RAG), StudyLink (Gemini RAG), Smart Research Assistant (early RAG)*

- [ ] RAG architecture: retriever + generator; why it reduces hallucination
- [ ] Document ingestion pipeline: PDF/text extraction, chunking strategy (fixed size, sentence, recursive)
  - → *Phoenix: RecursiveCharacterTextSplitter (800/150); StudyLink: 1000/200 character chunks with pypdf*
- [ ] Embedding models: sentence-transformers, OpenAI Ada, Gemini text-embedding; dimensionality trade-offs
- [ ] Vector similarity search: cosine similarity, inner product, L2 distance; HNSW vs. IVFFlat indexes
- [ ] Hybrid search: dense (vector) + sparse (BM25/TF-IDF) retrieval; score fusion strategies
  - → *Phoenix implements WLC hybrid fusion (70% vector + 30% BM25Okapi) with MinMaxScaler normalization*
- [ ] Retrieval quality measurement: precision@k, recall@k, NDCG; hit rate; MRR
- [ ] Confidence scoring and fallback: when to refuse vs. generate
  - → *Phoenix 4-tier state machine: CS >= 0.75 Direct, 0.5-0.75 Query expansion, 0.35-0.5 Cross-encoder reranking, < 0.35 Refusal*
- [ ] Cross-encoder reranking: FlashRank; ms-marco models; reranking vs. bi-encoder
  - → *Phoenix uses FlashRank ONNX cross-encoder for orange-tier retrieval fallback*
- [ ] Multi-tenant document isolation: scoping retrieval by `project_id` / `resource_id`
  - → *StudyLink scopes cosine similarity search strictly to `resource_id`; hard similarity cutoff at 0.65*
- [ ] Chunking quality: chunk size trade-offs; overlap; recursive splitting; semantic chunking
- [ ] Context window management: fitting retrieved chunks; token counting; truncation strategies

11.4 AI Agents & Multi-Agent Systems `[CORE for AI track]`

> *Reference: Vigil (LangGraph ReAct agent evaluation), Foundry (LangGraph multi-agent consensus), Conclave (multi-model pipeline), Trajectory (Spring AI extraction)*

- [ ] ReAct pattern: Reason + Act; tool calling loops; observation injection
  - → *Vigil evaluates LangGraph ReAct agents; `LangGraphAgentAdapter` enforces recursion limits (`max_steps * 2 + 2`)*
- [ ] LangGraph: `StateGraph`, nodes, edges, conditional edges, `add_node`, `add_edge`, `compile`
  - → *Vigil, Foundry use LangGraph; Foundry has Investor/PM/Tech Lead/Consistency/TieBreaker nodes*
- [ ] Tool/function calling: structured function schemas, tool routing, multi-tool agents
- [ ] Agent memory: episodic memory, semantic memory, external memory stores (database-backed)
  - → *Foundry's Decision Memory Engine persists structured design commitments to relational DB*
- [ ] Multi-agent patterns: sequential pipelines, debate loops, adversarial critique, consensus building
  - → *Conclave: sequential multi-model pipeline with mention-driven orchestration; Foundry: consensus + conflict detection*
- [ ] Canonical message schemas for multi-model systems: vendor-agnostic storage, runtime template compilation
  - → *Conclave stores all turns in `CanonicalMessage`; compiles to Llama/Mistral/Gemma templates at runtime*
- [ ] Context window management in multi-agent systems: history compression, "Context Janitor" pattern
  - → *Conclave's WorkflowStateServiceImpl compresses conversation to `WorkflowState` when > 10 messages*
- [ ] Agent evaluation: deterministic assertion-based vs. LLM-as-a-judge; reproducibility
  - → *Vigil provides the definitive reference: 6 assertion types, P50/P90 metrics, regression diffs*
- [ ] Safety and guardrails: prompt injection, tool sandboxing, resource limits, anomaly detection
  - → *Vigil: path traversal validation, subprocess allowlist scanner, loop detection, Docker isolation*
- [ ] LLMOps: versioning prompts, A/B testing agents, evaluation datasets, regression tracking
  - → *Vigil's RunComparator tracks task-level regressions (PASS to FAIL) across agent versions*

11.5 Spring AI `[STRETCH]`

> *Reference: Trajectory (Spring AI + Groq), Conclave (Spring AI + Ollama), Smart Research Assistant*

- [ ] Spring AI `ChatClient`: fluent API; model configuration; system/user prompt separation
- [ ] Prompt templates: `PromptTemplate`, placeholder substitution
- [ ] Model providers: OpenAI, Gemini, Anthropic, Ollama, Groq; configuration via `application.yml`

    - → *Trajectory uses `spring-ai-openai-spring-boot-starter:1.0.0-M1` routed to Groq API via base-url override*
    - → *Conclave uses Spring AI Ollama adapter for local Llama3/Mistral/Gemma*
- ☐ Structured output: `BeanOutputConverter`; `ChatClient.entity()`; Java records as schema
    - → *Trajectory maps AI responses to `JobExtraction`, `EventExtraction`, `OutreachAnalysis` records*
- ☐ Streaming: `stream().chatResponse()`, `Flux<ChatResponse>`; token-by-token streaming
- ☐ MCP (Model Context Protocol): connecting external tools to LLM context (concept)
- ☐ Embeddings in Spring AI: `EmbeddingModel`; vector store integration; `VectorStore.add/similaritySearch`
- ☐ Tool calling in Spring AI: `@Tool`, `ToolCallback`; tool result injection

11.6 Local LLM Deployment [STRETCH]

> *Reference: Phoenix (Ollama Mistral), Conclave (Ollama: Llama3, Mistral, Gemma)*

- ☐ Ollama: model pulling, serving, REST API (`/api/generate`, `/api/chat`); `model:tag` format
- ☐ Model families: Llama 3 (`<|start_header_id|>` templates), Mistral (`[INST]`), Gemma (`<start_of_turn>`)
- ☐ GGUF format: quantization levels (Q4_K_M, Q8_0); VRAM requirements; CPU fallback
- ☐ Cold-start latency: model weight loading; connection timeout configuration
    - → *Phoenix raises REST client timeout to 300s for Ollama Mistral cold-start on CPU-only hosts*
- ☐ Context window limits per model; multi-turn conversation token accumulation

---

Phase 12: QA & Specialized Testing

> ***Career relevance:*** *CORE for QA Engineer track; OPTIONAL for most backend/AI tracks (basic testing is in Phase 4/5).*

12.1 Testing Fundamentals (All Tracks) [CORE]

- ☐ Test pyramid: unit → integration → E2E; cost vs. confidence trade-offs at each level
- ☐ Unit testing principles: FIRST (Fast, Isolated, Repeatable, Self-validating, Timely)
- ☐ Test doubles: mock, stub, fake, spy, dummy; when each is appropriate
- ☐ Test-driven development (TDD): red-green-refactor cycle; when and when not to use
- ☐ Code coverage: statement, branch, condition coverage; why 100% coverage is not the goal
- ☐ Test isolation: database cleanup strategies; shared state problems

- → *Phoenix and Vigil both encountered DB test pollution; use `try-finally` teardown fixtures*

JUnit 5 & Mockito `[CORE for Java track]`

- [ ] `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`; `@Nested` test classes
- [ ] Assertions: `assertEquals`, `assertThrows`, `assertDoesNotThrow`, `assertAll`
- [ ] `@ExtendWith(MockitoExtension.class)`, `@Mock`, `@InjectMocks`, `@Spy`
- [ ] `when(...).thenReturn(...)`, `verify(...)`, `ArgumentCaptor`
- [ ] Spring Boot testing: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `MockMvc`, `@MockBean` (and `@MockitoBean` in Spring Boot 3.4+)
- [ ] Testcontainers: real PostgreSQL/Redis containers in integration tests

Pytest `[CORE for Python track]`

- [ ] `def test_*`, fixtures (`@pytest.fixture`), `conftest.py`, scope (`function`, `class`, `module`, `session`)
- [ ] `pytest.raises`, `pytest.mark.parametrize`, `pytest.mark.skip`
- [ ] Mocking with `unittest.mock`: `@patch`, `MagicMock`, `side_effect`
- [ ] `pytest-django`: `@pytest.mark.django_db`, Django test client, `APIClient`
- [ ] Concurrency testing: `ThreadPoolExecutor` in tests; detecting race conditions
  - → *Shard runs 20-50 parallel threads executing 500+ concurrent operations in stress tests*

12.2 Playwright E2E Testing `[CORE for QA track]`

> *Reference: Aegis (Playwright framework targeting Trajectory — 11 test spec files across functional, API, and visual regression suites), Foundry (Playwright E2E)*

- [ ] Playwright architecture: browser contexts, pages, `Page` API; Chromium, Firefox, WebKit
- [ ] Locator strategies priority: `getByRole` → `getByPlaceholder` → `getByLabel` → `data-testid` → CSS
  - → *Aegis enforces this locator priority order*
- [ ] Assertions: `expect(locator).toBeVisible()`, `toHaveText()`, `toHaveValue()`, `toBeEnabled()`
- [ ] Page Object Model (POM): encapsulating locators and actions; `BasePage` → specialized pages
  - → *Aegis: BasePage → LoginPage, DashboardPage, ApplicationPage, ResumePage*
- [ ] Custom fixtures: `test.extend()` for injecting page objects and API clients into tests
  - → *Aegis `test.fixture.ts` injects all page objects and API clients into test parameters*
- [ ] API client layer with Playwright: `APIRequestContext`, `request.post/get`, Bearer token injection
- [ ] Visual regression testing: `expect(page).toHaveScreenshot()`; baseline management; `maxDiffPixels`, `animations: 'disabled'`
  - → *Aegis configures visual tests with animation disabling and dynamic content masking*
- [ ] Headless vs. headed mode; browser configuration in `playwright.config.ts`
- [ ] Allure reporting: `allure-playwright`; step annotations; screenshot attachments
- [ ] Parallel execution: `workers`, shard testing across CI runners

- [ ] CI/CD integration: running Playwright in GitHub Actions; caching browser binaries

12.3 Performance & Load Testing `[CORE for QA track]`

> *Reference: Pulse (k6 framework targeting Phoenix/Trajectory)*

- [ ] Performance testing taxonomy: smoke, load, stress, soak, spike tests; what each measures
- [ ] k6: `export default function() {}`, `http.get/post`, `check()`, `sleep()`; `stages` VU ramping
  - → *Pulse uses load, stress, smoke, and recovery test scenarios with multi-stage VU ramps*
- [ ] k6 metrics: `http_req_duration` (p50, p90, p95, p99), `http_req_failed`, `vus`, `iterations`
- [ ] Thresholds: `thresholds: { 'http_req_duration': ['p95 < 500'] }` performance gates
- [ ] Performance budgets: p95/p99 latency budgets, error rate budgets; baseline comparison with 5% variance
  - → *Pulse implements baseline comparison with Improved/Regressed/Unchanged classification*
- [ ] InfluxDB + Grafana for k6 output: `--out influxdb`; pre-provisioned dashboards
- [ ] Load vs. stress testing: finding throughput capacity vs. finding breaking points
- [ ] Recovery testing: verifying system returns to normal after stress; `recovery_test.js`
- [ ] Interpreting results: p95 vs. p99 latency meaning; error rates; throughput saturation

12.4 Security Testing `[CORE for QA track]`

> *Reference: Warden (Python security suite with OWASP ZAP), Vigil (sandbox security)*

- [ ] OWASP Top 10: Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Authentication Failures, XSS, SSRF
- [ ] SQL Injection: error-based, time-based blind, boolean-based blind detection
  - → *Warden implements all 3 SQLi detection modes with database error pattern matching*
- [ ] XSS (Cross-Site Scripting): reflected, stored, DOM-based; detection via unescaped reflection
  - → *Warden scans for unescaped tag reflection in responses*
- [ ] IDOR (Insecure Direct Object Reference): resource ID manipulation; access control verification
  - → *Warden implements User A / User B access control model for IDOR testing*
- [ ] Authentication vulnerabilities: unprotected endpoints, bypass headers, weak session management
- [ ] Input fuzzing: boundary values, type mismatches, oversized inputs, malformed JSON; anomaly detection (HTTP 500)
  - → *Warden fuzzes parameters and detects unhandled server crashes as security anomalies*
- [ ] OWASP ZAP: daemon mode, REST API; spider scanning; active vs. passive scanning
  - → *Warden wraps ZAP API in a custom Python client; orchestrates baseline spider scans*
- [ ] Security findings: severity classification (Critical/High/Medium/Low); deduplication by signature
- [ ] Responsible disclosure: testing only authorized targets; safety boundaries
  - → *Warden enforces explicit authorization flag (`--authorized`) before any scan*

- [ ] Verify: Explain the difference between SQL injection and XSS; describe how to test for IDOR

12.5 API Testing `[CORE for QA track]`

- [ ] REST API testing: request construction, response validation (status, headers, body schema)
- [ ] API test isolation: setup and teardown; seeded test data; idempotent test operations
- [ ] Contract testing: consumer-driven contracts; Pact (concept)
- [ ] Authentication in tests: Bearer token injection; refresh token handling in test suites
- [ ] Using Playwright as an API client: `APIRequestContext`; multipart upload testing
  - → *Aegis `ResumesApi` tests multipart PDF upload as part of E2E resume verification*

Phase 13: Projects, Integration & Interview Preparation

13.1 Project Portfolio `[CORE]`

Existing projects provide practical verification of many checklist items. Use them to deepen understanding, not merely to list on a resume.

Project	Tech Stack	Key Concepts Demonstrated
Cairn	Java 21, Spring Boot 3.3.0, JUnit	JVM threading, `ReentrantReadWriteLock`, concurrent LRU engine, consistent hashing, Micrometer
Shard	Python, Django, DRF	CPython GIL mechanics, coarse-grained locking, O(1) LRU/LFU, consistent hashing, Prometheus
Phoenix	Spring Boot + FastAPI + React 19, pgvector	Hybrid RAG, confidence scoring, 4-tier fallback state machine, cross-encoder reranking
Vigil	Python, FastAPI, Docker SDK, LangGraph	Agent evaluation, Docker sandboxing, P50/P90 metrics, signal handling, deterministic assertions
Foundry	Django, Celery, LangGraph, React 19, Zustand	Multi-agent consensus, decision memory, ASGI, real-time streaming, conflict detection
Conclave	Java 21, Spring Boot, Spring AI, STOMP, React 19	Multi-model orchestration, STOMP streaming, canonical schema, pessimistic locking
Trajectory	Java 21, Spring Boot 3.3.4, Spring AI, React 19 + TS	Full-stack app tracking, JWT + OAuth2, Spring AI (Groq), S3, Docker Compose, AWS runbook
StudyLink	Django 5.1, DRF, pgvector, React 18	Scoped RAG, `select_for_update` state machine, custom user model, `transaction.on_commit()`
Check-Mate Analyse	React 19, TypeScript, Web Worker, WASM	Stockfish WASM, Web Worker thread isolation, UCI protocol, COOP/COEP headers
Aegis *(Testing)*	Playwright, TypeScript	POM framework, API testing layer, 11 spec files (functional + visual), CI pipeline, Allure

Project	Tech Stack	Key Concepts Demonstrated
Pulse *(Testing)*	k6, Docker, InfluxDB, Grafana	Load/stress testing, performance budgets, baseline comparison, observability stack
Warden *(Testing)*	Python, OWASP ZAP, Docker	SQLi/XSS/IDOR scanning, input fuzzing, report generation, ZAP API integration

13.2 Build New Projects [CORE]

For any tracks not already covered by existing projects, build 1-2 targeted projects:

- ☐ **Java Backend-only project** (if Cairn/Phoenix backend feels insufficient): Spring Boot REST API with Spring Security, Spring Data JPA, PostgreSQL, comprehensive JUnit + `@SpringBootTest` test suite
- ☐ **Python AI project**: FastAPI + LangGraph agent with tool calling, RAG retrieval, and deterministic evaluation harness
- ☐ **Full-stack project (deployed to production)**: React + Spring Boot/FastAPI + PostgreSQL, deployed with Docker + GitHub Actions + AWS

13.3 Debugging Skills [CORE]

- ☐ Docker container debugging: `docker logs`, `docker exec -it`, inspect network, check resource limits
  - → *Vigil: WSL2 memory overcommit nuances; ASGI misconfiguration; container cleanup on SIGINT*
- ☐ Database debugging: `EXPLAIN ANALYZE`, slow query log, lock contention via `pg_stat_activity`
- ☐ JVM debugging: thread dump analysis, heap dump, `jmap`, `jstack`, memory leak detection
- ☐ Python debugging: `pdb`, `breakpoint()`, logging at DEBUG level, tracing async code
- ☐ Network debugging: `curl`, `httpie`, Wireshark basics, TLS handshake failures
- ☐ Frontend debugging: React DevTools, Network tab, Performance profiler, Source Maps
- ☐ Verify: Deliberately introduce a race condition in Shard or Cairn, observe it, and fix it

13.4 Interview Preparation [CORE]

> *Full interview preparation checklist:* `D:\Work\Learning Engg\roadmap\checklist\Interview_Checklist.pdf`

Behavioral & Communication

- ☐ STAR method: Situation, Task, Action, Result for behavioral questions
- ☐ Technical storytelling: explain complex projects clearly to non-technical interviewers
- ☐ "Tell me about a challenge you faced" — prepare 3 project-specific stories (Vigil safety, Shard GIL, Phoenix retrieval)

DSA for Interviews

- [ ] Time-box problem solving: 5-min plan → code → test → optimize
- [ ] Verbalize thought process before writing code
- [ ] Handle edge cases: empty input, single element, duplicates, max values, null
- [ ] Complexity analysis: always state time and space complexity with reasoning
- [ ] Practice pattern → problem mapping: see keyword, identify pattern
- [ ] Complete the pattern matrix from `D:\Work\Learning Engg\Data-Structures-and-Algorithms\README.md`

System Design Interviews

- [ ] Framework: Clarify requirements → estimate scale → define API → sketch architecture → deep-dive components → trade-offs
- [ ] Always ask: read-heavy vs. write-heavy? consistency vs. availability? latency requirements?
- [ ] Practice HLD problems with 45-minute time limit
- [ ] Reference your own projects in interviews: Shard/Cairn for caching, Phoenix for RAG, Vigil for evaluation

Java/Spring Boot Technical Interviews

- [ ] Common question areas: JVM memory, GC, threading, Spring Boot bean lifecycle, JPA N+1, transaction propagation, Spring Security filter chain
- [ ] Code: implement LRU cache from scratch in Java without using `LinkedHashMap`
- [ ] Code: find all anagrams in a string using sliding window + hashmap

Python Technical Interviews

- [ ] GIL mechanics, thread-safety, `asyncio` event loop, decorator internals, generator vs. iterator
- [ ] Explain why `dict.update()` is thread-safe but `cache.update_and_evict()` is not

Coverage Notes & Source Grounding

The following sources were inspected when building this checklist:

Primary source of truth:

- `D:\Work\Learning Engg\roadmap\checklist\Technology_Checklist.pdf` (all 7 pages, 13 phases)

Learning repository files:

- `D:\Work\Learning Engg\Data-Structures-and-Algorithms\README.md` (Phase 2 pattern matrix)
- `D:\Work\Learning Engg\Java-Practice\README.md`
- `D:\Work\Learning Engg\Python-ChapterWise\README.md`

Project dossiers fully inspected:

- **Applied Projects (`Projects----For Resume`):** Vigil, Shard, Cairn, Phoenix, Foundry, Conclave, Trajectory, StudyLink, Check-Mate Analyse (and Email-Job-Scheduler in `Projects----Commercial\Outbox Labs\Assignment`)
- **DevSecOps & Testing Frameworks (`Projects----Testing`):** Aegis (README.md), Pulse (README.md), Warden (README.md, ROADMAP.md)
- **Architecture Runbooks & Training Dossiers:** Trajectory `infrastructure/aws-provisioning-guide.md` (documented design runbook), `D:\Work\Repo for Training Program\IBM-PBEL\INTERNSHIP_DOSSIER.md`, `D:\Work\Projects----Archived\Smart-Research-Assistant\README.md`

Topics added beyond original PDF based on project evidence:

- Docker SDK for Python (`docker-py`) and container sandbox engineering (Vigil)
- STOMP WebSocket protocol and Spring WebSocket (Conclave, Foundry)
- Multi-model LLM orchestration and canonical schemas (Conclave)
- Django Channels + Celery + Daphne ASGI stack (Foundry)
- pgvector HNSW indexing details (Phoenix, StudyLink)
- Hybrid search fusion: BM25 + vector, WLC, MinMaxScaler (Phoenix)
- Cross-encoder reranking with FlashRank ONNX (Phoenix)
- Spring AI with Groq + typed record binding (Trajectory)
- k6 performance testing framework and Pulse observability stack (Pulse)
- OWASP ZAP API integration and multi-scanner pipeline (Warden)
- Playwright API testing layer and Allure reporting (Aegis)
- Check-Mate Analyse: WASM, Web Workers, UCI protocol, COOP/COEP headers

> ***No source files were modified. Only `Master_Checklist.md` was created.***